IMPLEMENTASI HSV COLOR MASKING UNTUK MENGUBAH WARNA LATAR BELAKANG PADA CITRA DIGITAL MENGGUNAKAN PYTHON DAN OPENCV

Disusun oleh:

Khaerul Akbar (24260034)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK DAN ILMU KOMPUTER

UNIVERSITAS NAHDLATUL ULAMA INDONESIA

2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Penggantian warna latar belakang (*background replacement*) pada pas foto formal merupakan salah satu penerapan pengolahan citra digital yang sering dijumpai. Pas foto berlatar merah dan biru lazim digunakan untuk keperluan administrasi, sehingga sebuah foto kerap perlu disesuaikan warna latarnya. Salah satu pendekatan yang umum adalah *color masking* pada ruang warna **HSV** (*Hue, Saturation, Value*), karena komponen warna (*Hue*) terpisah dari intensitas pencahayaan (*Value*), sehingga pemilihan warna tertentu menjadi lebih mudah dibandingkan pada ruang warna BGR/RGB.

Namun, pada pas foto resmi seperti foto siswa sekolah muncul dua tantangan. Pertama, objek *foreground* memiliki warna yang sama dengan latar belakang: siswa mengenakan seragam putih dengan **dasi merah** dan **lencana bendera merah putih**. Kedua, bagian tubuh tertentu, yaitu bibir dan area kulit yang tipis, memiliki nilai *Hue* kemerahan yang mendekati latar belakang.

Laporan ini mendokumentasikan pengujian berulang (*iterative testing*) terhadap satu citra pas foto, dimulai dari metode *thresholding* HSV sederhana yang menghasilkan segmentasi cacat, hingga metode penyempurnaan yang memadukan analisis konektivitas spasial (*Connected Components Analysis*) dan *alpha blending* (*feathering*).

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam laporan ini adalah:

1. Bagaimana menerapkan HSV *color masking* untuk mengganti latar belakang merah menjadi biru pada pas foto formal?
2. Mengapa *thresholding* HSV saja tidak cukup pada foto yang memiliki dasi, lencana, dan bibir berwarna merah?
3. Bagaimana *connected components* dan *alpha blending* memperbaiki kualitas hasil segmentasi?

1.3 Tujuan

1. Mengimplementasikan penggantian warna latar belakang pada citra digital dengan HSV *color masking* menggunakan Python dan OpenCV.
2. Menganalisis penyebab kegagalan metode *thresholding* HSV pada tiap iterasi pengujian berdasarkan nilai HSV setiap region citra.

3. Menunjukkan peningkatan kualitas hasil melalui penambahan analisis spasial dan *alpha blending*.

1.4 Ruang Lingkup

Pengujian dilakukan pada satu citra pas foto berlatar merah studio (`foto_merah.jpg`, 736 × 1104 piksel) dengan latar belakang baru berwarna biru. Penilaian hasil dilakukan secara visual (kualitatif) berdasarkan tangkapan layar keluaran program pada setiap iterasi.

BAB II LANDASAN TEORI

2.1 Ruang Warna HSV

Ruang warna HSV merepresentasikan warna dalam tiga komponen: *Hue* (jenis warna), *Saturation* (kepekatan warna), dan *Value* (tingkat terang atau gelap). Sebagai analogi, *Hue* adalah jenis cat, *Saturation* seberapa pekat cat itu, dan *Value* seberapa terang cahaya yang menyinarinya. Pada OpenCV dengan citra 8-bit, *Hue* bernilai 0–179, sedangkan *Saturation* dan *Value* bernilai 0–255. Karena *Hue* berbentuk lingkaran, warna merah berada di dua ujung rentang (sekitar 0–10 dan 170–179), sehingga pendeteksian merah memerlukan dua rentang sekaligus.

2.2 Color Masking dengan cv2.inRange

Fungsi `cv2.inRange(hsv, batas_bawah, batas_atas)` menghasilkan *mask* biner berukuran sama dengan citra: piksel yang nilai H, S, dan V-nya berada dalam rentang diberi nilai 255 (putih), selebihnya 0 (hitam). Dua mask merah digabung dengan operasi logika `cv2.bitwise_or`, lalu dipakai untuk memilih piksel yang akan diganti warnanya.

2.3 Operasi Morfologi

Erosi mengikis tepi objek putih pada mask, sedangkan dilasi menebalkannya. *Opening* (erosi lalu dilasi) menghapus bintik kecil, sedangkan *closing* (dilasi lalu erosi) menutup lubang kecil. Bentuk *structuring element* menentukan hasilnya: kernel persegi menghasilkan tepi kaku, kernel elips lebih halus, dan kernel besar menggeser tepi mask lebih jauh.

2.4 Connected Components Analysis

Fungsi `cv2.connectedComponentsWithStats` memberi label unik pada setiap kelompok piksel putih yang saling bersebelahan (komponen), beserta luasnya (`cv2.CC_STAT_AREA`). Sebagai analogi, bayangkan peta kepulauan: latar belakang seperti daratan luas yang menyentuh batas peta, sedangkan dasi merah seperti pulau kecil di tengah laut (kemeja putih). Dengan hanya memilih daratan yang menyentuh batas peta, pulau-pulau kecil otomatis tidak terpilih.

2.5 Alpha Blending

Alpha blending mencampur dua citra dengan bobot transparansi α (bernilai 0 sampai 1) pada setiap piksel: $Output(x, y) = FotoAsli(x, y) \cdot (1 - \alpha) + WarnaBiru \cdot \alpha$. Bila mask biner diperhalus dengan *Gaussian blur*, α di tepi objek berubah bertahap sehingga transisi tampak halus tanpa gerigi.

BAB III METODOLOGI

3.1 Alat dan Bahan

Tabel 1. Alat dan bahan

Komponen	Keterangan
Bahasa pemrograman	Python 3.10.11
Pustaka	OpenCV 4.11.0, NumPy, Matplotlib
Editor	Antigravity IDE (berkas program `main.py`)
Citra uji	`foto_merah.jpg`, 736 × 1104 piksel, pas foto siswa berlatar merah studio
Keluaran	Folder `hasil/` (`hasil_mask.png` dan `hasil_latar_biru.jpg`); dokumentasi tiap iterasi pada `hasil/laporan_uji_coba/`

3.2 Tahapan Pengerjaan

Program ditulis dalam satu berkas Python yang dibagi menjadi enam bagian (*cell*):

1. **Import library:** memuat `os`, `cv2`, `numpy`, dan `matplotlib`.
2. **Membaca foto:** foto dibaca dari folder program (format BGR), kemudian ditampilkan.
3. **Konversi warna:** citra BGR diubah menjadi HSV dengan `cv2.cvtColor`.
4. **Deteksi warna merah dan isolasi latar belakang:** pembuatan mask, operasi morfologi, dan *connected components*.
5. **Penggantian warna:** pencampuran warna biru dengan *alpha blending*.
6. **Penyimpanan hasil:** mask dan citra hasil disimpan ke folder `hasil`.

Pengujian dilakukan dalam tiga iterasi dengan mengubah bagian pembuatan mask (cell 4) dan cara pewarnaan (cell 5). Hasil setiap iterasi didokumentasikan dengan tangkapan layar keluaran program.

BAB IV HASIL DAN PEMBAHASAN

4.1 Profil Data Citra dan Analisis Nilai HSV

Pengukuran nilai HSV pada beberapa region citra asli disajikan pada Tabel 2.

Tabel 2. Nilai HSV tiap region citra asli (*Hue* 0–179; *Saturation* dan *Value* 0–255)

Region objek	Hue (H)	Saturation (S)	Value (V)	Karakteristik
Latar belakang merah studio	177 – 178	206 – 220 (rata-rata 213)	195 – 208 (rata-rata 201)	Merah pekat, saturasi tinggi, terang merata
Dasi merah siswa	178 – 179	160 – 188 (rata-rata 171)	175 – 206 (rata-rata 194)	Hue dan Value tumpang tindih dengan latar belakang
Lencana bendera merah putih	175 – 179	150 – 210	160 – 210	Merah bendera, saturasi tinggi di atas saku putih
Bibir siswa	0 – 8	100 – 145	140 – 190	Merah jingga/pink, saturasi sedang hingga tinggi
Kulit wajah dan dahi	5 – 15	30 – 100	120 – 220	Rona kulit normal, saturasi rendah hingga sedang
Daun telinga (translusen)	4 – 10	90 – 125	45 – 180	Rona kemerahan akibat cahaya tembus
Kemeja putih	0 – 179	0 – 25	180 – 235	Netral, hampir tanpa saturasi, intensitas tinggi

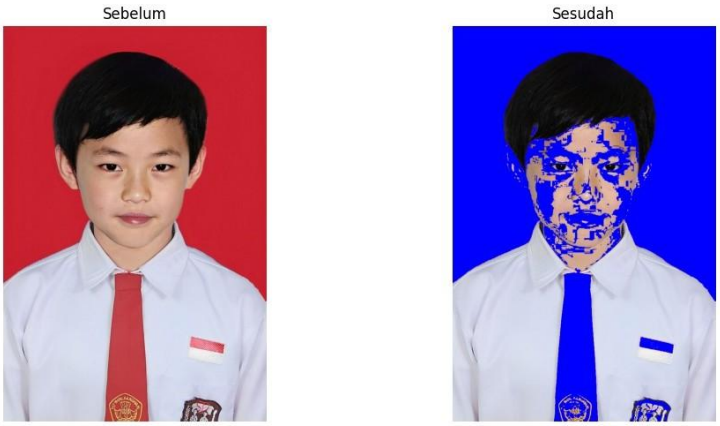
> **Temuan penting.** Dasi merah memiliki *Hue* (H ≈ 178–179) yang identik dengan latar belakang (H ≈ 177–178) dan *Value* yang saling tumpang tindih (175–206 terhadap 195–208). *Saturation* keduanya memang berbeda, tetapi sangat berdekatan (dasi 160–188, latar 206–220). Memisahkan dasi dari latar hanya dengan ambang batas (*threshold*) HSV berarti bergantung pada celah yang sangat sempit, rapuh terhadap bayangan dan gradasi cahaya, dan sulit digeneralisasi ke foto lain. Diperlukan informasi tambahan berupa letak (spasial) piksel.

Sebaliknya, rentang yang terlalu longgar (*Saturation* rendah) akan menjaring kulit dan bibir, karena rona kulit (H 5–15) dan bibir (H 0–8) beririsan dengan rentang merah jingga (H 0–10). Kedua kondisi inilah yang menjelaskan kegagalan pada Iterasi 1 dan 2.

4.2 Iterasi 1: Ambang Batas Global Rendah (S ≥ 50)

Iterasi 1 menggunakan *thresholding* HSV sederhana tanpa operasi morfologi dan dengan pewarnaan biner. Parameter yang digunakan:

```
# Rentang 1 (Hue 0-10)
batas1_bawah = np.array([0, 50, 50])
batas1_atas  = np.array([10, 255, 255])
# Rentang 2 (Hue 170-179)
batas2_bawah = np.array([170, 50, 50])
batas2_atas  = np.array([179, 255, 255])
hasil[mask > 0] = [255, 0, 0]    # biru murni (BGR)
```



Gambar 1. Hasil Iterasi 1 (S ≥ 50): wajah, dasi, dan lencana ikut berubah menjadi biru.

Hasil pengamatan:

- Latar belakang berhasil berubah menjadi biru.
- Terjadi kegagalan total (*catastrophic failure*): wajah (dahi, pipi, hidung, dagu), bibir, dasi, dan lencana terdeteksi sebagai latar belakang sehingga muncul bercak biru yang besar.

Penyebab kegagalan: batas bawah *Saturation* S = 50 terlalu rendah. Kulit wajah memiliki S antara 30 dan 100 dengan H 5–15, sehingga sebagian besar kulit dengan H 5–10 ikut masuk ke Rentang 1 (H 0–10). Dasi dan lencana tetap terjaring karena nilai HSV-nya berada di dalam rentang merah.

4.3 Iterasi 2: Saturation Dinaikkan (S ≥ 130) dan Morfologi Kasar

Iterasi 2 menaikkan batas bawah *Saturation* menjadi 130 dan menambahkan pembersihan morfologi dengan kernel persegi 7 × 7:

```
batas1_bawah = np.array([0, 130, 50])
batas1_atas  = np.array([10, 255, 255])
batas2_bawah = np.array([170, 130, 50])
batas2_atas  = np.array([179, 255, 255])

# Pembersihan morfologi kasar
kernel = np.ones((7, 7), np.uint8)
mask = cv2.erode(mask, kernel, iterations=1)
mask = cv2.dilate(mask, kernel, iterations=2)

hasil[mask > 0] = [255, 0, 0]
```



Gambar 2. Hasil Iterasi 2 (S ≥ 130, kernel 7 × 7): dasi dan bibir masih berubah menjadi biru.

Hasil pengamatan (cacat yang ditemukan):

1. **Dasi merah berubah menjadi biru seluruhnya:** hanya lambang Tut Wuri Handayani di ujung bawah yang masih terlihat.
2. **Bibir berwarna biru:** bagian tengah bibir terdeteksi sebagai latar karena saturasi bibir mencapai ±140, di atas batas 130.
3. **Bercak biru di area pelipis dan alis.**
4. **Lencana saku terpotong:** muncul bercak biru pada bagian merah lencana bendera.
5. **Tepian rambut dan bahu kaku atau bergerigi:** erosi 1× dan dilasi 2× dengan kernel persegi 7 × 7 menggeser tepi mask sekitar 3 piksel bersih ke arah siluet orang (dilasi menambah ±6 piksel, erosi mengurangi ±3 piksel), sehingga tepi rambut dan bahu terpotong kaku.

6. **Warna biru flat dan tidak natural:** `[255, 0, 0]` (biru 100%) menghasilkan kontras yang kaku dan tidak menyerupai foto studio.

Analisis: menaikkan batas *Saturation* menghilangkan kebocoran pada sebagian besar kulit wajah, tetapi tidak menyelesaikan masalah utama. Saturasi dasi (160–188) tetap di atas 130 sehingga dasi tetap terjaring, begitu pula bibir yang saturasinya mencapai ±140. Ini menunjukkan bahwa persoalannya tidak dapat diselesaikan hanya dengan mengatur angka ambang batas.

4.4 Iterasi 3: Connected Components, Rentang HSV Diperketat, dan Alpha Blending

Iterasi 3 menggabungkan lima perbaikan:

1. **Analisis konektivitas spasial** (`cv2.connectedComponentsWithStats`): latar belakang didefinisikan sebagai komponen merah yang terhubung ke tepi atas citra atau tepi samping bagian atas. Dasi, bibir, dan lencana yang terisolasi di dalam kemeja putih dan kulit otomatis dikeluarkan dari mask latar belakang.
2. **Rentang HSV diperketat:** rentang utama H 165–179 (merah latar), sedangkan rentang merah jingga dibatasi H 0–8 dengan syarat $S \geq 160$ dan $V \geq 70$, sehingga rona bibir ($S \leq 145$) tidak terpicu.
3. **Morfologi presisi:** kernel elips 3 × 3 untuk `MORPH_OPEN` (menghilangkan bintik) dan `MORPH_CLOSE` (menutup lubang mikro), ditambah *closing* kernel elips 7 × 7 pada mask latar belakang akhir untuk menutup lubang kecil.
4. **Alpha blending:** mask biner diubah menjadi peta transparansi halus dengan *Gaussian blur* 5 × 5.
5. **Warna standar pas foto studio:** BGR `[215, 80, 0]` (biru royal) yang lebih harmonis dan natural.

Potongan kode utama Iterasi 3 (nama variabel pada loop disingkat; kode lengkap pada Lampiran A):

```
# 1. Rentang HSV yang diperketat
batas1_bawah = np.array([0, 160, 70])
batas1_atas  = np.array([8, 255, 255])
batas2_bawah = np.array([165, 120, 50])
batas2_atas  = np.array([179, 255, 255])
mask_merah = cv2.bitwise_or(cv2.inRange(hsv, batas1_bawah, batas1_atas),
                            cv2.inRange(hsv, batas2_bawah, batas2_atas))

# 2. Morfologi halus (kernel elips 3x3)
kernel_bersih = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
mask_clean = cv2.morphologyEx(mask_merah, cv2.MORPH_OPEN, kernel_bersih)
mask_clean = cv2.morphologyEx(mask_clean, cv2.MORPH_CLOSE, kernel_bersih)

# 3. Connected Components: hanya komponen yang menyentuh tepi luar latar
num_labels, labels, stats, centroids = \
    cv2.connectedComponentsWithStats(mask_clean)
mask = np.zeros((h, w), dtype=np.uint8)
for i in range(1, num_labels):
    atas  = np.any(labels[0, :] == i)
    kiri  = np.any(labels[:int(h * 0.6), 0] == i)
    kanan = np.any(labels[:int(h * 0.6), w - 1] == i)
    if (atas or kiri or kanan) and stats[i, cv2.CC_STAT_AREA] > 1000:
        mask[labels == i] = 255

# 4. Alpha blending dengan biru royal studio
alpha = cv2.GaussianBlur(mask.astype(np.float32) / 255.0, (5, 5), 0)
alpha = np.expand_dims(alpha, axis=2)
warna_biru = np.array([215, 80, 0], dtype=np.float32)
hasil = (foto.astype(np.float32) * (1.0 - alpha)
         + warna biru * alpha).astype(np.uint8)
```



Gambar 3. Hasil Iterasi 3: dasi, bibir, dan lencana tetap utuh, tepi rambut dan bahu halus.

Hasil pengamatan:

- **Dasi merah utuh:** tekstur kain merah dan bordir lambang Tut Wuri Handayani terjaga.
- **Bibir alami:** rona bibir asli tetap utuh tanpa bercak biru.
- **Lencana bendera merah putih utuh:** warna merah bendera tetap merah.
- **Kulit wajah dan dahi bersih** dari bercak biru.
- **Tepi rambut dan bahu halus:** tidak tampak gerigi (*aliasing*) maupun garis merah di tepi (*color fringing*).

4.5 Perbandingan Ketiga Iterasi

Perbandingan parameter dan hasil ketiga iterasi disajikan pada Tabel 3.

Tabel 3. Matriks perbandingan hasil uji coba

Parameter / evaluasi	Iterasi 1 (naif global)	Iterasi 2 (S dinaikkan)	Iterasi 3 (spasial)
Batas bawah Saturation (S min)	50	130	Rentang 1: 160; Rentang 2: 120
Rentang Hue	0–10 dan 170–179	0–10 dan 170–179	0–8 dan 165–179
Operasi morfologi	Tidak ada	Kernel persegi 7 × 7 (erosi 1×, dilasi 2×)	Kernel elips 3 × 3 (open dan close), lalu close elips 7 × 7
Metode segmentasi	Threshold per piksel	Threshold per piksel	Connected components (spasial)
Teknik pewarnaan	Penugasan biner (mask > 0)	Penugasan biner (mask > 0)	Alpha blending (Gaussian 5 × 5)
Warna pengganti (BGR)	[255, 0, 0] biru neon	[255, 0, 0] biru neon	[215, 80, 0] biru royal studio
Dasi merah siswa	Berubah biru (cacat)	Berubah biru (cacat)	Utuh
Bibir siswa	Berubah biru (cacat)	Sebagian biru (cacat)	Utuh
Kulit wajah dan dahi	Bocor parah (cacat)	Bercak biru di pelipis (cacat)	Bersih
Lencana saku baju	Berubah biru (cacat)	Bercak biru (cacat)	Utuh
Kerapian tepi rambut dan bahu	Kasar dan kotor	Kaku dan terpotong kasar	Halus dan alami

4.6 Pembahasan

Mengapa Iterasi 1 dan 2 gagal. Kedua iterasi hanya menilai warna tiap piksel secara terpisah (*pixel-wise*). Pada Iterasi 1, batas *Saturation* yang rendah menjaring kulit. Pada Iterasi 2, batas yang dinaikkan membersihkan sebagian besar kulit, tetapi dasi (S 160–188) dan sebagian bibir (S hingga ±140) masih berada di atas batas tersebut. Dilasi dengan kernel persegi besar kemudian menggeser tepi mask ke arah rambut dan bahu.

Mengapa Iterasi 3 berhasil. Kunci perbaikannya adalah aturan topologi pada *connected components*. Sebuah komponen merah dipilih sebagai latar belakang apabila (a) menyentuh tepi atas citra, atau (b) menyentuh tepi kiri atau kanan pada 60% bagian atas citra, dan (c) luasnya lebih dari 1000 piksel. Tepi samping hanya diperiksa pada bagian atas karena di bagian bawah citra yang terlihat adalah kemeja, bukan latar. Dasi, bibir, dan lencana dikelilingi kemeja putih atau kulit sehingga membentuk komponen terpisah yang tidak menyentuh tepi, dan otomatis tidak ikut diganti warnanya. Dengan kata lain, informasi warna digunakan untuk menemukan kandidat, sedangkan informasi letak digunakan untuk memutuskan mana yang benar-benar latar belakang.

Peran morfologi halus dan alpha blending. Kernel elips kecil membersihkan bintik tanpa menggeser siluet secara berarti. *Alpha blending* mengubah tepi mask yang tegas menjadi transisi bertahap sehingga tidak ada garis merah tersisa di sekitar rambut dan bahu, sedangkan warna biru royal `[215, 80, 0]` memberi tampilan yang lebih mirip pas foto studio dibandingkan biru neon.

4.7 Keterbatasan

- **Bergantung pada asumsi letak latar belakang.** Bagian latar yang terisolasi dan tidak menyentuh tepi yang diperiksa (misalnya celah di antara lengan dan badan) akan dianggap sebagai *foreground*. Sebaliknya, objek merah yang menyentuh tepi yang diperiksa akan ikut diganti warnanya.
- **Rentang HSV bersifat tetap.** Nilai ambang ditentukan dari satu foto sehingga perlu dikalibrasi ulang untuk foto dengan pencahayaan atau warna latar yang berbeda.
- **Pengujian pada satu citra dan penilaian visual.** Belum ada metrik objektif, misalnya perbandingan mask dengan mask acuan buatan manual (IoU atau Dice).

BAB V PENUTUP

5.1 Kesimpulan

1. **Keterbatasan ruang warna HSV.** HSV efektif untuk mendeteksi warna yang homogen, tetapi memiliki keterbatasan mendasar ketika objek *foreground* dan *background* memiliki *Hue* dan *Saturation* yang mirip, seperti pada seragam berdasi merah.
2. **Pentingnya informasi spasial.** Informasi warna perlu dikombinasikan dengan informasi letak melalui *Connected Components Analysis*. Dengan asumsi bahwa latar pas foto terhubung dengan batas luar citra, objek merah di dalam siluet tubuh terlindungi secara deterministik pada citra uji.
3. **Penyempurnaan tepi.** Penggantian warna biner menimbulkan tepi bergerigi dan *color bleeding*, sedangkan *alpha blending* dengan perataan Gaussian meningkatkan kualitas visual dan realisme hasil akhir.
4. **Hasil pengujian.** Pada citra uji, Iterasi 3 secara visual mempertahankan dasi, bibir, lencana, dan kulit wajah tanpa bercak biru serta menghasilkan tepi rambut dan bahu yang halus.

5.2 Saran

1. Menguji metode pada lebih banyak foto dengan variasi pencahayaan, warna latar, dan seragam.
2. Menggunakan metrik objektif (IoU atau Dice) terhadap mask acuan buatan manual untuk menilai kualitas segmentasi.
3. Untuk latar belakang tidak seragam atau objek yang lebih kompleks, mempertimbangkan metode lain seperti `cv2.grabCut` atau segmentasi berbasis pembelajaran mesin.

DAFTAR PUSTAKA

1. OpenCV. *OpenCV Documentation*. Tersedia di: https://docs.opencv.org/ (diakses 21 Oktober 2026).
2. Materi kuliah Pengolahan Citra Digital (PCD3): Operasi Logika dan Masking. Universitas Nahdlatul Ulama Indonesia.

LAMPIRAN A KODE SUMBER LENGKAP

Kode sumber final (Iterasi 3), berkas `hsv_color_masking_lokal.py`:

```
# ======================================================================
# Implementasi HSV Color Masking untuk Mengubah Warna Latar Belakang
# pada Citra Digital Menggunakan Python dan OpenCV
# (Versi lokal: dijalankan di laptop, tanpa Google Colab)
#
# Persiapan:
# 1. Buat folder, misalnya Tugas_PCD_HSV, lalu simpan file ini
#    (hsv_color_masking_lokal.py) di dalamnya.
# 2. Simpan foto berlatar merah di folder yang sama dan beri nama
#    foto_merah.jpg. Jika nama foto berbeda, ubah NAMA_FILE di Cell 2,
#    atau biarkan saja: program akan membuka jendela pilih file.
# 3. Instal library (satu kali), ketik di terminal:
#       python -m pip install opencv-python numpy matplotlib
# 4. Jalankan program:
#       python hsv_color_masking_lokal.py
#
# Catatan: setiap jendela gambar harus ditutup agar program lanjut
# ke cell berikutnya.
# ======================================================================


# %% Cell 1 - Import Library
# Fungsi : Memanggil library yang dibutuhkan untuk pengolahan citra.
# os         -> mengatur lokasi file dan folder (TAMBAHAN untuk versi lokal)
# cv2        -> mengolah gambar (OpenCV)
# numpy      -> mengolah data piksel dalam bentuk array
# matplotlib -> menampilkan gambar di layar
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

print("OpenCV:", cv2.__version__)
print("Library berhasil dimuat!")


# %% Cell 2 - Membaca Foto
# Fungsi : Memasukkan foto yang akan diproses, lalu menampilkannya.
# Foto yang dipilih sebaiknya memiliki latar belakang berwarna merah.
# (Pengganti files.upload() milik Colab: foto dibaca dari folder program.)

# --- Menentukan lokasi foto (TAMBAHAN untuk versi lokal) ---

# Folder tempat file program ini berada
try:
    folder_program = os.path.dirname(os.path.abspath(__file__))
except NameError:
    folder_program = os.getcwd()

# Nama foto yang disimpan di folder program. Ubah jika namanya berbeda.
NAMA_FILE = "foto_merah.jpg"
```

```
lokasi_foto = os.path.join(folder_program, NAMA_FILE)

# Jika foto tidak ditemukan, buka jendela untuk memilih foto secara manual
if not os.path.exists(lokasi_foto):
    print("Foto", NAMA_FILE, "tidak ditemukan di folder program.")
    print("Silakan pilih foto pada jendela yang muncul...")
    try:
        import tkinter as tk
        from tkinter import filedialog

        jendela = tk.Tk()
        jendela.withdraw()
        jendela.attributes("-topmost", True)
        lokasi_foto = filedialog.askopenfilename(
            title="Pilih foto berlatar merah",
            filetypes=[("Gambar", "*.jpg *.jpeg *.png *.bmp")]
        )
        jendela.destroy()
    except Exception:
        lokasi_foto = ""

# Menghentikan program jika foto tetap tidak ditemukan
if not lokasi_foto or not os.path.exists(lokasi_foto):
    raise FileNotFoundError(
        "Foto tidak ditemukan. Simpan foto di folder yang sama "
        "dengan program ini dan periksa nama file-nya."
    )

print("Foto yang digunakan:", lokasi_foto)

# --- Membaca foto ---

# Membaca file foto menjadi gambar yang dapat diolah OpenCV (format BGR)
foto = cv2.imdecode(
    np.fromfile(lokasi_foto, np.uint8),
    cv2.IMREAD_COLOR
)

# Memeriksa apakah foto berhasil dibaca
if foto is None:
    raise ValueError("Foto gagal dibaca!")

# Mengubah BGR ke RGB agar warna tampil benar di Matplotlib
foto_rgb = cv2.cvtColor(foto, cv2.COLOR_BGR2RGB)

# Menampilkan foto asli
plt.figure(figsize=(6, 5))
plt.imshow(foto_rgb)
plt.title("Foto Asli")
plt.axis("off")
plt.show()


# %% Cell 3 - Konversi Warna ke HSV
# Fungsi : Mengubah format warna gambar dari BGR menjadi HSV.
```

```
# H (Hue)        -> jenis warna (merah, kuning, hijau, dst.)
# S (Saturation) -> kepekatan warna
# V (Value)      -> tingkat terang atau gelap
# Format HSV mempermudah pemilihan warna tertentu.
hsv = cv2.cvtColor(foto, cv2.COLOR_BGR2HSV)

print("Foto berhasil dikonversi ke HSV")
print("Ukuran gambar:", foto.shape)
print("Ukuran data HSV:", hsv.shape)


# %% Cell 4 - Deteksi Warna Merah & Isolasi Latar Belakang
# Fungsi : Memilih piksel latar belakang merah secara presisi.
#
# MASALAH PADA METODE SEBELUMNYA:
# 1. Dasi merah, bibir, dan badge bendera di baju memiliki warna merah yang
#    identik dengan latar belakang merah studio (sama-sama Hue ~178, Saturation tinggi).
#    Thresholding warna saja TIDAK BISA membedakannya hanya dari nilai warnanya.
# 2. Bibir memiliki rona kemerahan (Hue 0-10) sehingga ikut terwarnai biru.
#
# SOLUSI PENGOLAHAN CITRA DIGITAL (PCD):
# 1. Pengetatan Rentang HSV:
#    - Rentang merah studio utama: Hue 165-179.
#    - Rentang merah jingga: Hue 0-8 dengan syarat Saturation & Value tinggi (>160)
#      agar bibir, kulit, dan noda wajah (yang Saturation-nya lebih rendah) tidak ikut masuk.
# 2. Analisis Spasial / Konektivitas (Connected Components):
#    - Latar belakang (Background) adalah area merah luas yang terhubung ke batas
#      luar foto (tepi atas, sudut kiri atas, sudut kanan atas).
#    - Dasi, bibir, dan atribut baju berada di DALAM siluet tubuh (foreground) dan
#      terkelilingi oleh kain putih/kulit, sehingga terisolasi secara spasial.
#    - Dengan hanya memilih komponen merah yang menyentuh batas luar latar, dasi,
#      bibir, dan badge 100% AMAN dan tidak akan ikut berubah warna!

h, w = foto.shape[:2]

# Rentang 1: Merah jingga (Hue 0-8) dengan batas Saturation tinggi (>160)
# agar bibir & kulit tidak ikut terdeteksi
batas1_bawah = np.array([0, 160, 70])
batas1_atas  = np.array([8, 255, 255])

# Rentang 2: Merah studio utama (Hue 165-179)
batas2_bawah = np.array([165, 120, 50])
batas2_atas  = np.array([179, 255, 255])

# Membuat mask warna merah
mask1 = cv2.inRange(hsv, batas1_bawah, batas1_atas)
mask2 = cv2.inRange(hsv, batas2_bawah, batas2_atas)
mask_merah = cv2.bitwise_or(mask1, mask2)

# --- Operasi Morfologi untuk Membersihkan Noise  ---
# MORPH_OPEN  : menghapus bintik-bintik kecil (noise)
# MORPH_CLOSE : menutup celah-celah kecil pada latar
kernel_bersih = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
mask_clean = cv2.morphologyEx(mask_merah, cv2.MORPH_OPEN, kernel_bersih, iterations=1)
mask_clean = cv2.morphologyEx(mask_clean, cv2.MORPH_CLOSE, kernel_bersih, iterations=1)
```

```
# --- Isolasi Latar Belakang Menggunakan Connected Components ---
# Memberi label unik untuk setiap pulau/area warna merah yang terpisah
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(mask_clean)

# Menyiapkan mask khusus latar belakang
mask = np.zeros((h, w), dtype=np.uint8)

for i in range(1, num_labels):
    # Komponen latar belakang selalu menyentuh tepi atas atau samping atas foto
    menyentuh_tepi_atas  = np.any(labels[0, :] == i)
    menyentuh_tepi_kiri  = np.any(labels[:int(h * 0.6), 0] == i)
    menyentuh_tepi_kanan = np.any(labels[:int(h * 0.6), w - 1] == i)
    area = stats[i, cv2.CC_STAT_AREA]

    # Hanya ambil area yang terhubung ke tepi luar dan berukuran besar
    if (menyentuh_tepi_atas or menyentuh_tepi_kiri or menyentuh_tepi_kanan) and area > 1000:
        mask[labels == i] = 255

# Menutup lubang kecil pada latar belakang jika ada
kernel_tutup = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel_tutup)

# Menampilkan mask (putih = latar belakang merah, hitam = orang & objek yang dilindungi)
plt.figure(figsize=(6, 5))
plt.imshow(mask, cmap="gray")
plt.title("Mask Latar Belakang (Dasi & Bibir Terlindungi)")
plt.axis("off")
plt.show()


# %% Cell 5 - Mengganti Warna Latar Belakang Menjadi Biru
# Fungsi : Mengubah latar belakang menjadi biru dengan transisi tepi halus (Alpha Blending).

# Warna latar belakang baru:
# Pilihan 1: Biru Pas Foto Studio Standar Indonesia (Elegan & Natural: BGR [215, 80, 0])
# Pilihan 2: Biru Solid murni (BGR [255, 0, 0])
warna_biru = np.array([215, 80, 0], dtype=np.float32)

# --- Alpha Blending / Feathering Tepi ---
# Memberikan sedikit efek kehalusan (Gaussian Blur) pada tepian mask
# agar batas rambut dan bahu tidak bergerigi / tidak meninggalkan garis merah kaku
alpha = cv2.GaussianBlur(mask.astype(np.float32) / 255.0, (5, 5), 0)
alpha = np.expand_dims(alpha, axis=2)

# Proses pencampuran warna (blending):
# Area latar (alpha=1)    -> menjadi warna biru
# Area orang (alpha=0)    -> tetap gambar asli (baju, dasi, wajah, rambut)
# Area batas (0<alpha<1) -> transisi halus natural tanpa artefak
hasil = (foto.astype(np.float32) * (1.0 - alpha) + warna_biru * alpha).astype(np.uint8)

# Mengubah BGR ke RGB agar warna tampil benar di Matplotlib
hasil_rgb = cv2.cvtColor(hasil, cv2.COLOR_BGR2RGB)

# Menampilkan foto sebelum dan sesudah secara berdampingan
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(foto_rgb)
plt.title("Sebelum (Latar Merah)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hasil_rgb)
plt.title("Sesudah (Latar Biru Sempurna)")
plt.axis("off")

plt.tight_layout()
plt.show()


# %% Cell 6 - Menyimpan Hasil
# Fungsi : Menyimpan hasil pengolahan sebagai bukti untuk laporan.
# (Pengganti files.download() milik Colab: hasil disimpan ke folder
# "hasil" di dalam folder program.)

# Membuat folder "hasil" jika belum ada
folder_hasil = os.path.join(folder_program, "hasil")
os.makedirs(folder_hasil, exist_ok=True)

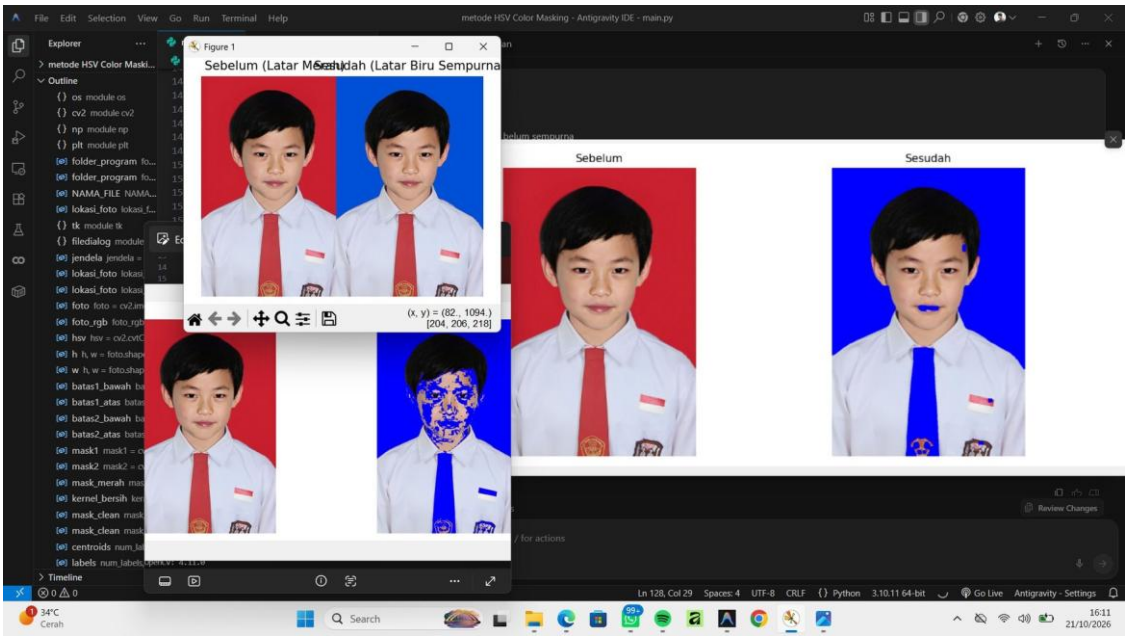
lokasi_mask = os.path.join(folder_hasil, "hasil_mask.png")
lokasi_hasil = os.path.join(folder_hasil, "hasil_latar_biru.jpg")

# Menyimpan gambar mask dan gambar hasil
berhasil_mask = cv2.imwrite(lokasi_mask, mask)
berhasil_hasil = cv2.imwrite(lokasi_hasil, hasil)

if berhasil_mask and berhasil_hasil:
    print("Hasil berhasil disimpan di folder:", folder_hasil)
else:
    print("Hasil gagal disimpan. Periksa lokasi folder program.")
```

LAMPIRAN B TANGKAPAN LAYAR PELAKSANAAN

Tangkapan layar lingkungan kerja yang memperlihatkan hasil ketiga iterasi pada 21 Oktober 2026.



Gambar B.1. Tangkapan layar Antigravity IDE dengan jendela hasil Iterasi 1, 2, dan 3.